

第七章 指令微调

指令微调 (Instruction Tuning) 是指使用**自然语言形式的数据**对预训练后的大语言模型进行参数微调，这一术语由谷歌研究员在 2022 年的一篇 ICLR 论文中正式提出 [39]。在另外一些参考文献中，指令微调也被称为有监督微调 (Supervised Fine-tuning) [28] 或多任务提示训练 (Multitask Prompted Training) [40]。指令微调过程需要首先收集或构建指令化的实例，然后通过有监督的方式对大语言模型的参数进行微调。经过指令微调后，大语言模型能够展现出较强的指令遵循能力，可以通过零样本学习的方式解决多种下游任务。在本章中我们将介绍指令数据的构建方法 (第 7.1 节) 和相应的训练策略 (第 7.2 节)，然后介绍低资源场景下的参数高效微调方法 (第 7.3 节)，最后给出指令微调和参数高效微调的代码实践与实验结果分析 (第 7.4 节)。

7.1 指令数据的构建

一般来说，一个经过指令格式化的数据实例包括任务描述 (也称为指令)、任务输入-任务输出以及可选的示例。在第 3 章中介绍了代表性的指令数据集合 (参考表 3.3)。下面，我们将主要介绍三种构建格式化指令数据的方法，并进一步讨论指令数据构造过程中需要考虑的重要因素。

7.1.1 基于现有的 NLP 任务数据集构建

学术界围绕传统 NLP 任务 (如机器翻译、文本摘要和文本分类等) 发布了大量的开源数据集，这些数据是非常重要的监督学习数据资源，可以用于指令数据集的构造。通常来说，这些 NLP 数据集都包括输入和输出两个主要部分。例如，在中英翻译任务中，输入是“大语言模型已经成为机器学习的一个重要研究方向”，而相应的输出则是“*Large language models have become one important research direction for machine learning*”。为了生成指令化的训练数据，一个非常关键的步骤就是为上述的“输入-输出”对数据添加任务描述信息，用于指导模型去理解任务目标以及相关信息。在上述的例子中，可以向中译英的翻译数据集中添加指令，例如“请把这个中文句子翻译成英文”。通过上述操作，就可以将一个 NLP 任务

的数据实例全部通过自然语言形式进行表达，进而数据实例可以被用于大语言模型的指令微调。

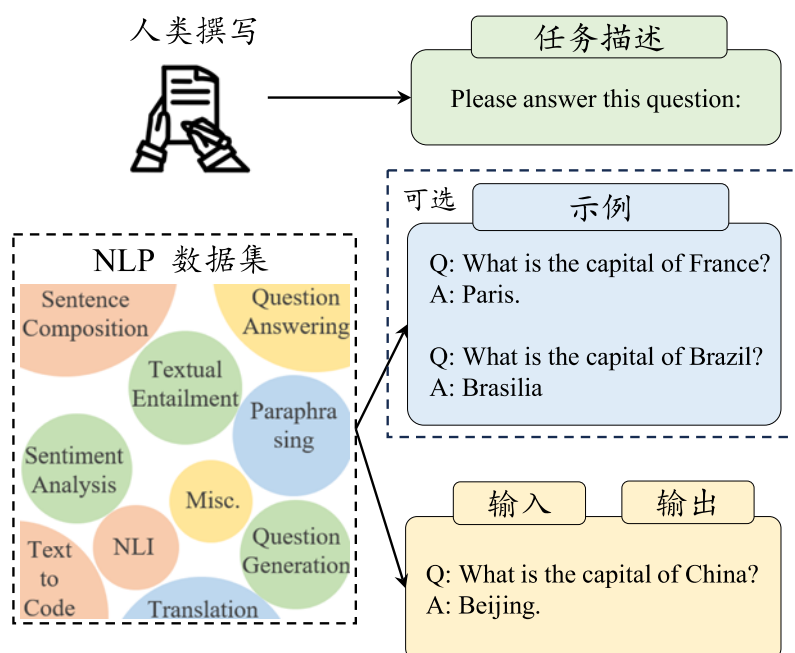


图 7.1 现有 NLP 数据集的指令格式化示意图 (图片来源: [10])

经过 NLP 指令数据微调后，大语言模型可以学习到指令遵循 (Instruction Following) 的能力，进而能够解决其他未见过的 NLP 任务 [39–41]。相关研究表明在现有 NLP 数据集的输入-输出数据中添加合适的任务描述是提升大模型指令跟随能力的关键因素；如果去除了这些任务描述，仅使用输入-输出数据对模型进行微调，模型的性能会出现显著下降 [39]。为了更好地标注 NLP 指令微调数据，研究人员开发了众包平台 PromptSource¹，它能够高效地支持标注人员为不同数据集创建、共享及验证任务描述。此外，为了进一步丰富训练实例，还可以设计特定的任务描述来翻转现有实例的输入-输出对，从而得到新的任务数据用于指令微调。例如，给定一个问题-答案对，传统的训练方式是基于问题预测答案；同时也可以基于答案生成问题，以此方式来创建一个新的“问题生成”实例，比如可以使用任务描述“请基于以下答案生成一个问题:”)。

在第 3 章中，表 3.3 汇总了常见的 NLP 任务数据集，如 FLAN、P3、Super-Natural Instructions 等。进一步，谷歌的研究人员按照一定的比例混合了这三个数据集和思维链数据，推出了包含约 20M 条实例的指令微调数据集 FLAN v2 [41]，该数据集目前是面向 NLP 任务微调大模型的首选指令集合。

¹<https://github.com/bigscience-workshop/promptsources>

7.1.2 基于日常对话数据构建

尽管通过指令格式化已有的 NLP 数据集能够获得大量的指令数据实例，但是这些数据的多样性比较局限，与人类的真实需求也不能很好匹配。为此，研究人员开始使用用户在日常对话中的实际需求作为任务描述。例如，InstructGPT [28] 将用户提交给 OpenAI API 的查询作为任务描述。由于这些用户查询源自于真实应用场景，均采用自然语言形式进行表达，因此特别适合大模型学习指令跟随能力。为了进一步增加任务的多样性，OpenAI 还雇佣标注者创作更多的真实生活任务，包括开放式生成、开放式问答、头脑风暴等任务，然后由另一组标注者来回答这些问题作为输出。OpenAI 最终将指令（用户真实查询或者人工标注的任务）和期望的输出（人工编写的答案）配对作为一个训练实例。但是，OpenAI 没有对外开放所使用的指令数据。

用户第一轮输入：激活函数有什么作用？

模型第一轮输出：激活函数是神经网络中的一种非线性函数，通常应用于神经元的输出上。它的作用是为神经网络引入非线性特性，使得神经网络可以更好地逼近复杂的非线性函数。如果不使用激活函数，神经网络将只能实现线性变换，无法学习复杂的非线性关系。……

用户第二轮输入：什么是复杂的非线性关系？

模型第二轮输出：复杂的非线性关系指的是输入与输出之间存在着非常复杂的非线性映射关系，无法用简单的线性函数来描述。这样的关系可能是高维度、非平稳、非对称等。例如，在图像识别任务中，一张图片中的不同物体之间的关系非常复杂，无法用简单的线性函数来描述。……

……（省略后续多轮对话）

例 7.1 ShareGPT 中的一个多轮日常对话数据

为了推动指令微调数据集的开放共享，最近研究人员创建并且开源了一些常见的日常对话任务指令集，例如 Dolly [105] 和 OpenAssistant [104]，这些数据均由人工精心标注，因此具有较高的质量。此外，研究人员还尝试通过自行构建的开放平台收集大量的用户对话请求作为输入数据，并使用 ChatGPT 或 GPT-4 生成回复作为输出数据。其中，一个代表性的数据集是来自于 ShareGPT 的多轮对话数据 [38]。

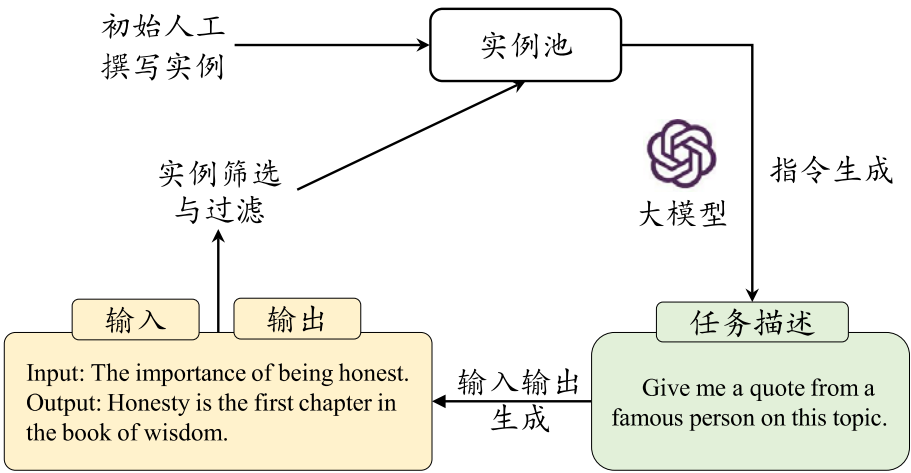


图 7.2 合成数据的指令形式示意图（图片来源：[10]）

7.1.3 基于合成数据构建

为了减轻人工收集与标注数据的负担，研究人员进一步提出半自动化的数据合成方法。他们借助已有的高质量指令数据作为上下文学习示例输入大语言模型，进而生成大量多样化的任务描述和输入-输出数据 [74, 198]。如图 7.2 所示，代表性工作 Self-Instruct [74] 方法仅需要使用 100 多个人工撰写的实例作为初始任务池，然后随机选择数据作为示例，就可以通过提示大语言模型生成新的指令微调数据。这种半自动化的合成方法具备高效生成大规模指令微调数据的能力，从而显著降低了人工标注所需的经济成本，在实践中得到了广泛应用。

考虑到 Self-Instruct 生成的实例可能过于简单或缺乏多样性，WizardLM [198] 进一步提出了一种改进的指令数据合成方法 Evol-Instruct，该方法通过基于深度和广度的演化来提高实例的复杂性和多样性。此外，Self-Align [199] 设计了多种基于人类对齐原则的合成数据过滤技术，该方法通过上下文提示让 ChatGPT 能够筛选出高质量的实例数据来训练新的模型，并进一步让新训练的模型产生更多与人类对齐的指令微调数据。与此同时，还有研究团队采用了指令回译技术 [200] 来优化输出文本的质量：他们直接使用现有的文本（例如维基网页数据）作为输出，然后利用上下文学习来逆向合成相应的输入指令。由于输出内容都是人工撰写的，该方法能够让模型学习生成准确且流畅的文本。

为了帮助读者理解这部分内容，这里介绍两个典型的指令数据合成技术：

Self-Instruct

Self-Instruct [74] 方法借助大语言模型（例如 ChatGPT）所具备的数据合成能

力，通过迭代的方法高效地生成大量的指令微调数据。作为初始任务池，该方法首先构建了 175 条高质量且多样的指令数据，之后经由两个主要步骤生成指令微调数据。

(1) 指令数据生成. 从任务池中随机选取少量指令数据作为示例，并针对 ChatGPT 设计精细指令来提示模型生成新的微调数据。具体地，ChatGPT 模型将下图中的指令和上下文示例，来仿照生成一些新的任务描述和对应的输出：

你被要求提供 10 个多样化的任务指令。这些任务指令将被提供给 GPT 模型。

以下是你提供指令需要满足的要求：

1. 尽量不要在每个指令中重复动词，要最大化指令的多样性。
2. 使用指令的语气也应该多样化。例如，将问题与祈使句结合起来。

……（省略后续要求）

下面是 10 个任务指令的列表：

指令：将 85 华氏度转换为摄氏度。

输出：85 华氏度等于 29.44 摄氏度。

指令：是否有科学无法解释的事情？

输出：有很多科学无法解释的事情，比如生命的起源、意识的存在……

……（省略上下文示例）

例 7.2 Self-Instruct 指令示例

(2) 过滤与后处理. 该步骤的主要目的是剔除低质量或者重复的生成实例，从而保证指令数据的多样性与有效性。常见的过滤方法包括：去除与任务池中指令相似度过高的指令、语言模型难以生成回复的指令、过长或过短的指令以及输入或输出存在重复的实例。

Self-Instruct 目前已经成为一种合成指令的基础方法，原始论文使用 GPT-3 合成了 Self-Instruct-52K 数据集，Alpaca 进一步使用了能力更强的 text-davinci-003 合成了 Alpaca-52K 数据集，之后研究人员陆续采用了更强大的模型（例如 GPT-4）来合成各种语言、各种领域的指令数据。

Evol-Instruct

WizardLM [198] 所提出的 Evol-Instruct 方法是一种基于大语言模型的指令数

据复杂化技术。该方法基于初始指令数据集（例如，Alpaca 指令数据集）进行扩展，主要包含两个步骤：

(1) 指令演化. 在该步骤中，大语言模型作为指令演化器，针对两个不同的方向进行指令的拓展，分别为深度演化和广度演化。深度演化通过五种特定类型的提示（添加约束、深化、具体化、增加推理步骤以及使输入复杂化）使得指令变得更加复杂与困难；而广度演化旨在扩充指令主题范围、指令涉及的能力范围以及整体数据集的多样性。

我希望您充当指令重写器。

您的目标是将给定的提示重写为更复杂的版本，使得著名的 AI 系统（例如 Chat-GPT 和 GPT-4）更难处理。

但重写的提示必须是合理的，且必须是人类能够理解和响应的。

您的重写不能省略 # 给定提示 # 中表格和代码等非文本部分。

您应该使用以下方法使给定的提示复杂化：

请在 # 给定提示 # 中添加一项约束或要求。

你应该尽量不要让 # 重写提示 # 变得冗长，# 重写提示 # 只能在 # 给定提示 # 中添加 10 到 20 个单词。

重写提示 # 中不允许出现“# 给定提示 #”、“# 重写提示 #”字段。

给定提示 #: {需要重写的指令}

重写提示 #:

例 7.3 Evol-Instruct 深度演化（添加约束）指令

例 7.3 和 7.4 分别是深度演化（添加约束）和广度演化的具体指令，我们将此提示和需要重写的指令输入到大语言模型中，模型便会根据这些指令生成演化后新的提示。然后再将这个指令输入给大模型来得到相应的答案，这样便构建了一条新的指令-输出数据实例。

我希望你充当指令创造器。

您的目标是从 # 给定提示 # 中汲取灵感来创建全新的提示。

此新提示应与 # 给定提示 # 属于同一领域，但更为少见。

创造提示 # 的长度和复杂性应与 # 给定提示 # 类似。

创造提示 # 必须合理，并且必须能够被人类理解和响应。

创造提示 # 中不允许出现“# 给定提示 #”、“# 创造提示 #”字段。

给定提示 #: {需要重写的指令}

创造提示 #:

例 7.4 Evol-Instruct 广度演化指令

(2) 数据后处理. 该阶段将去除部分实例数据以保证数据集合的整体质量和多样性。主要使用了如下的规则进行处理：使用 ChatGPT 比较演化前后的指令，移除 ChatGPT 认为差异很小的指令；移除大模型难以响应的指令，如响应中包含“sorry”或响应长度过短；移除仅包含标点符号和连词的指令或回复。

Evol-Instruct 主要使用了 OpenAI 的大语言模型 gpt-3.5-turbo 进行指令演化，该方法现在也被应用到了数学 (WizardMath)、代码 (WizardCoder) 等领域的数据合成中去，来增强这些领域数据的深度和广度。

7.1.4 指令数据构建的提升方法

在指令微调中，指令数据的格式、数量等因素对微调后的模型性能有着重要影响。下面将从指令格式设计、扩展指令数量、指令重写与筛选等三个方面介绍如何构建高质量的指令数据集。

指令格式设计

指令格式是影响大模型性能的一个重要因素 [201]。通常来说，可以直接向现有 NLP 数据集的输入-输出对上添加任务描述构建指令微调数据，这其中任务描述是大模型理解任务的关键部分。此外，还可以引入适当数量的实例作为上下文示例一起作为模型的输入，提升模型的实际性能，缓解模型对于指令格式的敏感性。FLAN-T5 的训练过程中同时使用了带示例的指令数据（即少样本）和不带示例的指令数据（即零样本）。实验发现，这种混合提示的训练方式有助于同时改善

下游任务中少样本和零样本的测试效果。

为了激发大模型的逐步推理能力，研究人员还尝试在指令微调数据集中引入思维链数据 [41]，例如算术推理的逐步解答过程。FLAN-T5 和 FLAN-PaLM 在指令微调时同时引入了包含 CoT 和不包含 CoT 的实例，通过这种混合指令数据微调后的模型在多种下游任务中都取得了较好的效果，包括需要多跳推理能力的任务（例如常识问答和算术推理）以及不需要多跳推理的任务（例如情感分析和抽取式问答） [41, 202]。

然而，指令数据并不是包含信息越多越好，添加某些看似有用的信息（例如需要避免的事项、原因或建议）到指令中，可能不会带来明显的效果提升，甚至可能产生不利影响 [201]。

扩展指令数量

对于 NLP 任务数据集而言，FLAN-T5 [41] 研究了指令数量对于模型在未知 NLP 任务上的性能影响。通过逐步将指令数量扩展至 0.18M、5.55M、7.2M 以及 17.26M，研究人员发现模型性能呈持续上升的趋势。然而，当指令数量达到 7.2M 后，模型性能的提升变得非常缓慢。

进一步，InstructGPT [28] 的研究工作发现，FLAN-T5 中采用的指令可能仅对传统 NLP 任务适用，而对于日常对话这一至关重要的能力并未带来明显的提升。实际上，对于一个较好的预训练基座模型，越来越多的研究工作表明一定数量的高质量指令就可以激活大语言模型的对话能力。Alpaca [42] 使用了 52K 条合成数据来指令微调 LLaMA (7B)，在 179 条日常对话数据的评测中到达了接近 text-davinci-003 的效果。进一步，LIMA [203] 仅使用了一千条人工标注的高质量指令数据来微调 LLaMA (65B)，就在 300 条日常对话测试中取得了较好的模型表现。

但是，仅依靠少量指令数据难以兼顾 NLP 任务和对话场景任务。Orca [204] 尝试基于 FLAN-T5 的大规模指令集进行扩展，让 GPT-4 和 ChatGPT 逐步回答每一个问题，得到了 5M 条合成实例，可以用于加强下游模型的逐步推理能力。以此数据微调的 LLaMA (13B) 能够同时在面向 NLP 任务（AGIEval）与日常对话任务（VicunaBench）的测试中同时取得较好的水平。

指令重写与筛选

面对众多的公开指令数据集，研究者们开始尝试使用一些重写或者筛选机制，来提高指令数据的质量或者多样性。Evol-Instruct 中使用了“深度演化”和“广度

演化”策略来重写指令，使指令变得更加困难或者多样。YuLan-Chat-3 [73] 提出了“主题多样性”增强方法，预先从知乎收集了 293 种常见主题标签（例如，“教育”，“体育”），然后随机选择一种并使用 ChatGPT 对指令进行重写来适配到相应的主题（例如使用提示：“请帮我把以下指令重写为教育主题”），最后进行质量筛选来获取高质量的多样性指令数据。

另一方面，研究人员尝试从大量数据集中筛选出部分指令来进行微调。Alpaga-sus 利用 GPT-4 从原始的 52K 条 Alpaca 数据中筛选出 GPT-4 评分较高的 9 千条。使用这 9 千条数据进行指令微调训练，便可以在下游测试任务中达到与原始 52K 条数据接近的效果。YuLan-Chat-3 提出了“平衡指令难度”策略，其用大模型的困惑度分数来估算指令数据的难度水平，删除过于简单或过于困难的指令数据，从而缓解大模型训练不稳定或者过拟合的现象。

总结

总体来说，指令的质量比数量更为重要。指令微调中应该优先使用人工标注的多样性指令数据。然而，如何大规模标注符合人类需求的指令数据目前仍然缺乏规范性的指导标准（比如什么类型的数据更容易激发大模型的能力）。在实践中，可以使用 ChatGPT、GPT-4 等闭源大语言模型来合成、重写、筛选现有指令，并通过数量来弥补质量和多样性上的不足。

7.1.5 指令微调的作用

在这一部分中，我们主要从三个方面讨论指令微调对大语言模型的影响。

整体任务性能改进

指令微调旨在使用人工构建的指令数据对于大语言模型进一步训练，从而增强或解锁大语言模型的能力 [41]。相关研究表明，不同规模的语言模型（参数量规模从 77M 到 540B）都可以从指令微调中受益 [41, 103]，提升的性能随着参数规模的增加而提升 [205]。此外，经过指令微调的小模型甚至可以比没有经过微调的大模型表现得更出色，进一步凸显了指令微调的有效性 [40, 41]。除了参数规模外，指令微调在不同的模型架构（编码器-解码器和因果解码器）、预训练目标（语言建模和去噪自编码）和模型微调方法（序列到序列损失和混合去噪器）上都能取得相对稳定的增益 [41]，这说明指令微调是一种非常通用的模型能力增强方法 [41]。同时，与预训练相比，指令微调的成本显著降低，大模型所需的指令数据量仅为

预训练阶段的约万分之一甚至更少。

任务求解能力增强

指令微调旨在指导模型学会理解自然语言指令，并据此完成相应的任务。通过指令微调，大模型能够获得较好的指令遵循与任务求解能力，无需下游任务的训练样本或者示例就可以解决训练中未见过的任务。指令微调还可以缓解预训练阶段大模型会出现的一些常见问题，例如生成重复内容或者仅仅补全输入而不解决相关任务 [28, 41]。此外，使用英文指令微调数据训练的大模型还可以将相应的能力泛化到其他语言的相关任务上。例如，BLOOM 是一个多语言的预训练语言模型 [100]，研究人员使用纯英文任务集合 P3 对其进行指令微调得到 BLOOMZ-P3 模型 [205]。有趣的是，在多语言句子补全任务中，BLOOMZ-P3 相比于基座模型 BLOOM 表现提升超过 50%。这些实验结果表明，指令微调能够帮助大模型从纯英文数据中获得较为通用的任务解决能力，并将这些能力迁移到其他语言 [205]。

领域专业化适配

通用的大语言模型能够在传统自然语言处理任务（如生成和推理）以及日常生活任务（如头脑风暴）上取得较好的效果，然而它们在特定领域中（如医学、法律和金融等）的表现与领域专用模型的效果仍有一定差距。在实际应用中，可以针对大语言模型进行面向特定领域的指令微调，从而使之能够适配下游的任务。以医学领域为例，研究人员提出使用医学数据集对 FLAN-PaLM [41] 进行微调，得到了医学知识助手模型 Med-PaLM [206]，其性能水平可与专业临床医生相媲美；国内研究学者也开源了基于 LLaMA 指令微调后的医学模型，例如“本草” [207]。在电子商务领域，研究人员也针对大模型进行微调，从而使之适配于推荐系统中的多种任务 [208]，取得了出色的效果提升。与此同时，研究人员还在法律、金融等领域探索了指令微调大模型的适配性 [209, 210]。这些工作表明，指令微调为大模型提供了一种通用的领域适配方法，拓宽了它们在实际场景中的应用范围。我们将在第 13 章针对大模型在不同领域的应用进行更为详细的讨论。

7.2 指令微调的训练策略

在训练方式上，指令微调与预训练较为相似，很多设置包括数据组织形式都可以预训练阶段所采用的技术（参考第 4 章和第 6 章）。本节主要介绍指令微调所特有的一些训练策略。

7.2.1 优化设置

指令微调中的优化器设置（AdamW 或 Adafactor）、稳定训练技巧（权重衰减和梯度裁剪）和训练技术（3D 并行、ZeRO 和混合精度训练）都与预训练保持阶段一致，可以完全沿用。下面主要介绍一些指令微调与预训练的不同之处。

- **目标函数.** 预训练阶段通常采用语言建模损失（详见第 6.1.1 节），优化模型在每一个词元上的损失。而指令微调可以被视为一个有监督的训练过程，通常采用的目标函数为序列到序列损失，仅在输出部分计算损失，而不计算输入部分的损失。

- **批次大小和学习率.** 考虑到预训练阶段已经学习到了能够展现较好性能的模型参数，指令微调阶段通常只需要使用较小的批次大小和学习率对模型进行小幅度的调整。例如 InstructGPT (175B) 微调的批次大小为 8，学习率恒定为 5.03×10^{-6} ；Alpaca (7B) 微调的批次大小为 128，学习率预热到 2×10^{-5} ，然后采用余弦衰减策略。

- **多轮对话数据的高效训练.** 对于一个多轮对话数据，通常的训练算法是将其拆分成多个不同的对话数据进行单独训练。为了提升训练效率，可以采用特殊的掩码机制来实现多轮对话数据的高效训练。在因果解码器架构中，由于输入输出没有明显的分界，可以将所有一个对话的多轮内容一次性输入模型，通过设计损失掩码来实现仅针对每轮对话的模型输出部分进行损失计算，从而显著减少重复前缀计算的开销。如例 7.1 所示，多轮对话涉及多次用户输入和模型的输出，但是训练中仅需要在模型的输出上计算损失。

7.2.2 数据组织策略

除了这些优化参数的设置，指令微调过程中还需要考虑一定的数据组织形式，从而使得模型获得更好的微调效果。下面介绍三种常用的数据组织策略。

平衡数据分布

现有的单一指令数据集（如表 3.3 中）通常只能增强大语言模型某些方面的能力，而无法提升模型的全方位能力 [211]。因此，研究者通常建议混合使用现有的多个指令数据集，以此来实现模型能力的综合改进。最常见的方法是样本比例混合策略 [77]，即把所有数据集进行合并，然后从混合数据集中等概率采样每个实例。例如，研究者建议混合使用 NLP 任务数据（如 FLAN v2）、对话数据（如

ShareGPT) 和合成数据 (如 GPT4-Alpaca), 来进行大模型的指令微调。

进一步地, 研究工作 [41, 202] 表明提高高质量数据集 (例如 FLAN 和 P3) 的采样比例通常可以带来性能提升。在 FLAN v2 数据集中, 最终使用的混合比例为: 46% 的 FLAN, 27.9% 的 T0, 24.2% 的 NIV2 和 1.8% 的 CoT 数据。为了避免数量较大的数据集主导整个采样过程, 指令微调过程中通常会设置一个最大容量, 用来限制每个数据集中可以采样的最大实例数 [77]。在实践中, 最大容量通常设置为几千或几万个实例 [39, 41]。例如, FLAN v2 中 FLAN、T0、NIV2 和 CoT 集合的最大容量分别设置为 30,000、20,000、5,000 和 100,000。

多阶段指令数据微调

第 7.1 节中介绍了三种常用的指令微调数据, 包括 NLP 任务数据、日常对话数据和合成数据。由于这些指令数据数量不同且内容差异较大 (表 3.3), 需要在训练中对于这些数据资源进行有效的调度, 进而达到较好的训练效果。为此, YuLan-Chat-3 采用了“多阶段指令微调”策略: 首先使用大规模 NLP 任务指令数据对模型进行微调, 然后再使用相对多样的日常对话指令和合成指令进一步微调。为了避免能力遗忘问题, 可以在第二阶段中添加一些 NLP 指令数据。这种多阶段的微调策略也可以应用于其他训练设置。例如, 对于不同的微调阶段, 训练中可以逐渐增加指令的难度和复杂性, 从而逐渐提高大模型遵循复杂指令的能力。

结合预训练数据与指令微调数据

为了使得微调过程更加有效和稳定, 可以在指令微调期间引入了预训练数据和任务, 这可以看作是对于指令微调的正则化。OPT-IML [202] 在指令微调阶段引入了 5% 的预训练数据, 在分类和生成任务上都能取得增益; 然而, 进一步增加预训练数据会对生成任务有利, 但有可能损失分类任务的表现。在另一方面, 将指令数据引入到预训练阶段也成为了一种常见的训练技术。通过提前使用指令微调数据, 有可能会帮助模型在预训练阶段更好地感知下游任务, 从而更为针对性地从预训练数据中学习知识与能力。例如, GLM-130B [162] 的预训练过程由 95% 的传统自监督预训练和 5% 的指令微调任务混合组成。MiniCPM [72] 提出在预训练阶段和指令微调阶段之间添加一个“退火阶段”, 该阶段混合使用高质量的预训练数据和指令微调数据, 其实验结果表明该策略优于先预训练再指令微调的两阶段策略。

7.3 参数高效的模型微调

在上述章节中已经深入探讨了指令微调的各种策略。通过指令微调，大语言模型能够更好地学习遵循和执行人类指令。然而，由于大语言模型的参数量巨大，进行全参数微调（需要较多的算力资源开销。在本节中，我们将讨论如何针对大语言模型进行参数高效微调（Parameter-efficient Fine-tuning），也称为轻量化微调（Lightweight Fine-tuning）。在现有文献中，参数高效微调 [212–214] 是一个重要的研究方向，旨在减少需要训练的模型参数量，同时保证微调后的模型性能能够与全量微调的表现相媲美。下面将首先介绍常用于 Transformer 架构的参数高效微调方法，然后以 LoRA 微调方法为例介绍参数高效微调的代码实现。

7.3.1 低秩适配微调方法

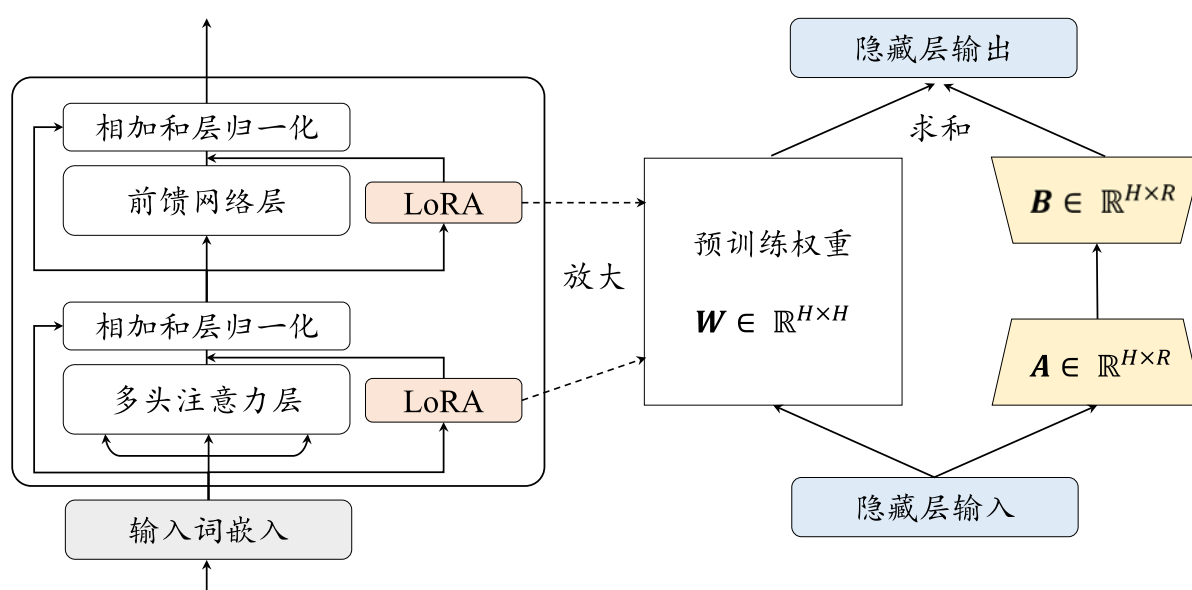


图 7.3 LoRA 微调示意图

本节中，我们首先介绍基础的低秩适配（Low-Rank Adaptation, LoRA）微调技术，然后介绍它的一些变种和在大模型场景下的应用。

LoRA 基础

大语言模型中包含大量的线性变换层（详见第 6.4.2 节），其中参数矩阵的维度通常很高。研究人员 [214] 发现模型在针对特定任务进行适配时，参数矩阵往往是过参数化（Over-parametrized）的，其存在一个较低的内在秩。为了解决这一问题，LoRA [214] 提出在预训练模型的参数矩阵上添加低秩分解矩阵来近似每层的

参数更新，从而减少适配下游任务所需要训练的参数。给定一个参数矩阵 $\mathbf{W}$ ，其更新过程可以一般性地表达为以下形式：

$$\mathbf{W} = \mathbf{W}_0 + \Delta\mathbf{W}, \quad (7.1)$$

其中， $\mathbf{W}_0$ 是原始参数矩阵， $\Delta\mathbf{W}$ 是更新的梯度矩阵。LoRA 的基本思想是冻结原始矩阵 $\mathbf{W}_0 \in \mathbb{R}^{H \times H}$ ，通过低秩分解矩阵 $\mathbf{A} \in \mathbb{R}^{H \times R}$ 和 $\mathbf{B} \in \mathbb{R}^{H \times R}$ 来近似参数更新矩阵 $\Delta\mathbf{W} = \mathbf{A} \cdot \mathbf{B}^\top$ ，其中 $R \ll H$ 是减小后的秩。在微调期间，原始的矩阵参数 $\mathbf{W}_0$ 不会被更新，低秩分解矩阵 $\mathbf{A}$ 和 $\mathbf{B}$ 则是可训练参数用于适配下游任务。在前向传播过程中，原始计算中间状态 $\mathbf{h} = \mathbf{W}_0 \cdot \mathbf{x}$ 的公式修改为：

$$\mathbf{h} = \mathbf{W}_0 \cdot \mathbf{x} + \mathbf{A} \cdot \mathbf{B}^\top \cdot \mathbf{x}. \quad (7.2)$$

在训练完成后，进一步将原始参数矩阵 $\mathbf{W}_0$ 和训练得到的权重 $\mathbf{A}$ 和 $\mathbf{B}$ 进行合并： $\mathbf{W} = \mathbf{W}_0 + \mathbf{A} \cdot \mathbf{B}^\top$ ，得到更新后的参数矩阵。因此，LoRA 微调得到的模型在解码过程中不会增加额外的开销。

LoRA 所需的显存估计

在第 6.4.4 节中，我们已经分析了全量微调场景下需要的显存大小，这里继续使用前文的计算方法来估算 LoRA 微调节省的显存资源，此处以不使用 ZeRO 技术为例。这里假设 LoRA 需要训练的参数量为 P_{LoRA} ，模型原始参数为 P 。考虑到模型参数与优化器是显存占用的主要部分，这里主要考虑它们的大小，其它显存占用部分与第 6.4.4 节几乎一致且占比较小，因此忽略不计。LoRA 微调需要保存的模型参数量为 $2P + 2P_{\text{LoRA}}$ ，梯度和优化器参数总计 $2P_{\text{LoRA}} + 4P_{\text{LoRA}} + 4P_{\text{LoRA}} + 4P_{\text{LoRA}} = 14P_{\text{LoRA}}$ ，因此 LoRA 微调需要的显存大小从全量微调的 $16P$ 大幅减少为 $2P + 16P_{\text{LoRA}}$ 。一般来说，LoRA 主要被应用在每个多头注意力层的 4 个线性变换矩阵上（即 $\mathbf{W}^Q, \mathbf{W}^K, \mathbf{W}^V, \mathbf{W}^O \in \mathbb{R}^{H \times H}$ ），因此 $P_{\text{LoRA}} = 4 \cdot 2 \cdot L \cdot HR$ ， L, H, R 分别是模型层数、中间状态维度和秩。以 LLaMA (7B) ($L = 32, H = 4096$) 为例，常见的秩 R 设置为 8，则 $P_{\text{LoRA}} = 8,388,608$ ， $2P + 16P_{\text{LoRA}} = 13,611,048,960 = 14\text{GB}$ ， $16P = 107,814,649,856 = 108\text{GB}$ 。可以看到，模型微调需要的显存大小从 108GB 大幅下降到 14GB，能够有效减少微调模型所需要的硬件资源。考虑到 $P_{\text{LoRA}} \ll P$ ，可以近似地认为轻量化微调需要的显存从 $16P$ 降至 $2P$ 。

LoRA 变种

在原始的 LoRA 实现中，每个低秩矩阵的低秩参数 R 都被设置为固定且相同的数值，并且在训练过程中无法进行调整，这种设定忽略了不同的秩在微调任务

中可能产生的差异化影响。因此，通过这种方式训练得到的低秩矩阵往往并非最优解。AdaLoRA [215] 讨论了如何更好地进行秩的设置。它引入了一种动态低秩适应技术，在训练过程中动态调整每个参数矩阵需要训练的秩同时控制训练的参数总量。具体来说，模型在微调过程中通过损失来衡量每个参数矩阵对训练结果的重要性，重要性较高的参数矩阵被赋予比较高的秩，进而能够更好地学习到有助于任务的信息。相对而言，不太重要的参数矩阵被赋予比较低的秩，来防止过拟合并节省计算资源。尽管 LoRA 能够有效地节省显存，但对于参数规模达到上百亿级别的模型而言，其微调所需的成本仍然相当高昂。QLoRA [216] 将原始的参数矩阵量化为 4 比特，而低秩参数部分仍使用 16 比特进行训练，在保持微调效果的同时进一步节省了显存开销。根据上一小节的分析，对于给定参数量为 P 的模型，QLoRA 微调所需要的显存由 LoRA 微调所需要的 $2P$ 进一步下降为 $0.5P$ 。因此通过 QLoRA 技术，可以在一张 A6000 (48GB) 的 GPU 上微调 65B 的模型，接近 16 比特模型微调的性能。

LoRA 在大语言模型中的应用

随着大语言模型的兴起，LoRA 这种参数高效的微调方法受到越来越多的关注。相关研究工作 [217] 对于参数高效微调方法进行了广泛的分析，并在超过一百个自然语言处理任务上，全面对比了全参数微调与现有的各类参数高效微调方法（第 7.3.2 节中介绍）。与全参数微调相比，LoRA 微调在保证模型效果的同时，能够显著降低模型训练的成本，被广泛地应用于开源大语言模型（如 LLaMA 和 BLOOM）的参数高效微调中。例如，Alpaca-LoRA² 是经过 LoRA 训练出的 Alpaca 模型的轻量化微调版本，它基于 7B 参数的 LLaMA 模型在 52K 条合成数据上进行了 LoRA 微调。进一步地，开源社区使用 LoRA 对于 LLaMA 系列模型进行了广泛探索，覆盖了不同语言（中文、泰语、西班牙语等）、不同领域的适配（对话、数学、代码等）。总体而言，LoRA 在各种高效微调方法中表现相对较好，需要训练的参数数量较少，并且易于实现，得到了较为广泛的应用。

7.3.2 其他高效微调方法

在本部分内容中，我们继续介绍其他的高效微调方法，包括适配器微调、前缀微调、提示微调。这三种方法在预训练语言模型中被广泛使用，但是在

²<https://github.com/tloen/alpaca-lora>

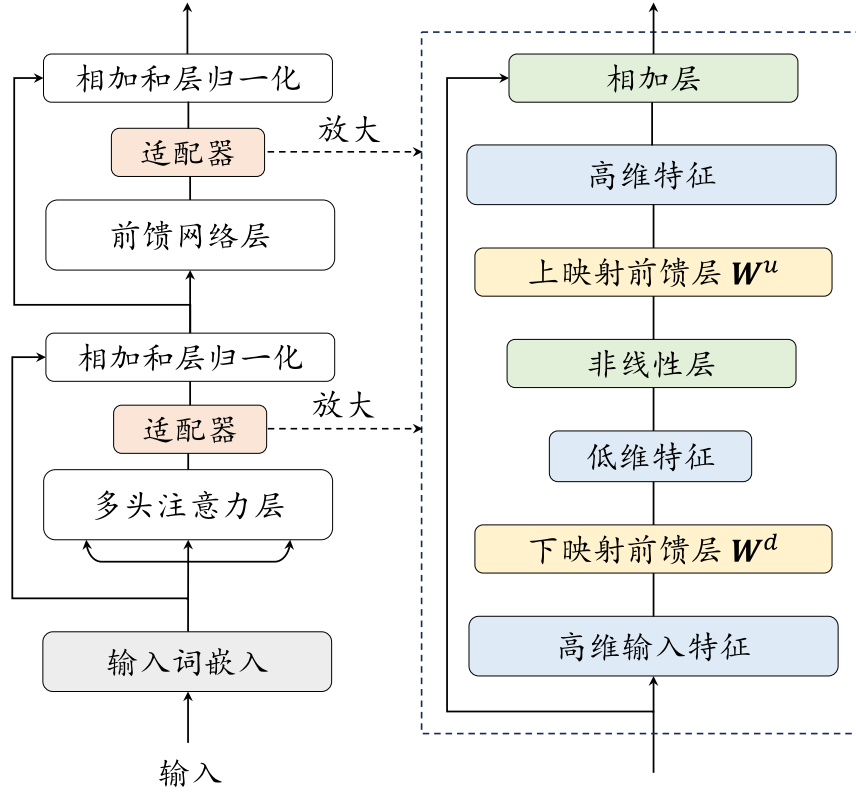


图 7.4 适配器微调示意图

模型中的应用相对较少。为了内容完整性，本书也引入了对这三种方法的介绍。

适配器微调

适配器微调 (Adapter Tuning) 在 Transformer 模型中引入了小型神经网络模块 (称为适配器) [218]。为了实现适配器模块，研究者提出使用瓶颈网络架构：它首先将原始的特征向量压缩到较低维度，然后使用激活函数进行非线性变换，最后再将其恢复到原始维度。形式化地，可以通过以下公式进行表达：

$$\mathbf{h} = \mathbf{h} + \sigma(\mathbf{h} \cdot \mathbf{W}^d) \cdot \mathbf{W}^u, \quad (7.3)$$

其中 $\mathbf{W}^d \in \mathbb{R}^{H \times R}$, $\mathbf{W}^u \in \mathbb{R}^{R \times H}$, 且 $R \ll H$ 。通常来说，适配器模块将会被集成到 Transformer 架构的每一层中，使用串行的方式分别插入在多头注意力层和前馈网络层之后、层归一化之前。在微调过程中，适配器模块将根据特定的任务目标进行优化，而原始的语言模型参数在这个过程中保持不变。通过这种方式，可以在微调过程中有效减少需要训练参数的数量。图 7.4 展示了适配器微调算法的示意图。

前缀微调

前缀微调 (Prefix Tuning) [212] 在语言模型的每个多头注意力层中都添加了一组前缀参数。这些前缀参数组成了一个可训练的连续矩阵，可以视为若干虚拟词

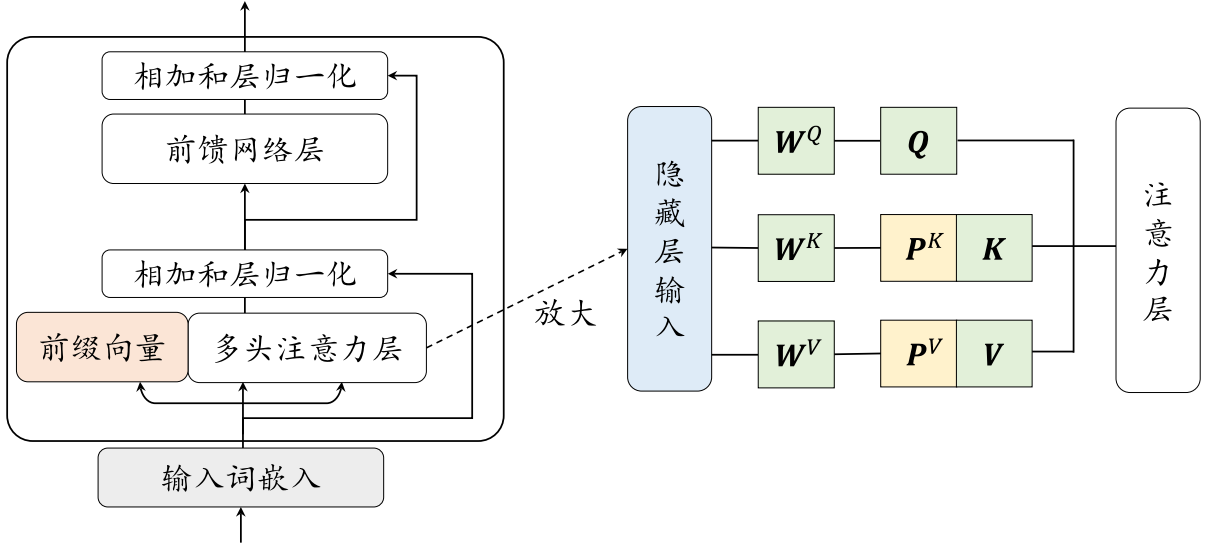


图 7.5 前缀微调示意图

元的嵌入向量，它们会根据特定任务进行学习。在具体实现上，基于原始的注意力计算公式 5.7，一系列前缀词元被拼接每个注意力的键向量与值向量之前，每个头的计算公式可以表示如下：

$$\text{head} = \text{Attention}(XW^Q, P^K \oplus XW^K, P^V \oplus XW^V), \quad (7.4)$$

其中 Attention 代表原始的注意力操作， $\oplus$ 表示矩阵拼接， $P^K, P^V \in \mathbb{R}^{L \times H}$ ， L 代表前缀向量的长度，一般在 10 到 100 之间，可以根据任务场景进行调整。为了更好地优化前缀向量，研究者提出了一种重参数化技巧 [212]，他们引入了一个多层感知机的映射函数 $P = \text{MLP}_\theta(P')$ 。重参数化技巧可以将较小的矩阵映射到前缀参数矩阵，而不是直接优化前缀。经实验证明，这一技巧对于稳定训练很有帮助。经过优化后，映射函数将被舍弃，只保留最终得到的前缀参数 P 来增强特定任务的性能。在前缀微调过程中，整个模型中只有前缀参数会被训练，因此可以实现参数高效的模型优化。图 7.5 展示了前缀微调算法的示意图。

提示微调

与前缀微调不同，提示微调 [213, 219] 仅在输入嵌入层中加入可训练的提示向量。在离散提示方法的基础上（将在第 10.1 章中详细介绍），提示微调首先在输入文本端插入一组连续嵌入数值的提示词元，这些提示词元可以以自由形式 [219] 或前缀形式 [213] 来增强输入文本，用于解决特定的下游任务。在具体实现中，只需要将可学习的特定任务提示向量与输入文本向量结合起来一起输入到语言模型中。P-tuning [219] 提出了使用自由形式来组合输入文本和提示向量，通过双向 LSTM

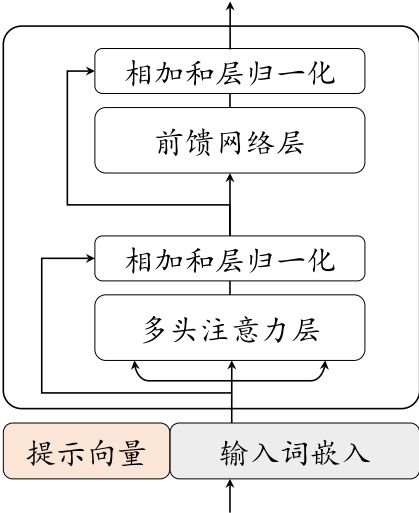


图 7.6 提示微调示意图

来学习软提示词元的表示，它可以同时适用于自然语言理解和生成任务。另一种代表性方法称为 Prompt Tuning [213]³，它以前缀形式添加提示，直接在输入前拼接连续型向量。在提示微调的训练过程中，只有提示的嵌入向量会根据特定任务进行监督学习，然而由于只在输入层中包含了极少量的可训练参数，有研究工作表明该方法的性能高度依赖底层语言模型的能力 [213]。图 7.6 展示了提示微调算法的示意图。

7.4 代码实践与分析

为了帮助读者更好地理解指令微调和参数高效微调的具体实现，本节将给出相应的示例代码，并基于该代码测试不同指令数据对于模型微调性能的表现。

7.4.1 指令微调的代码实践

指令微调的示例代码与预训练的代码（详见第 6.5 节）高度一致，区别主要在于指令微调数据集的构建（SFTDataset）和序列到序列损失的计算（DataCollatorForSupervisedDataset）。以下代码展示了 LLMBox 和 YuLan-Chat 中指令微调的整体训练流程。

³本书用提示微调泛指一类通过在提示端引入额外参数进行微调的方法，而使用英文表达 Prompt Tuning 表示一种特定的提示微调方法 [213]。

```

1 import torch
2 from dataclasses import dataclass
3 from dataset.sft_dataset import SFTDataset
4 from transformers import (
5     AutoModelForCausalLM,
6     AutoTokenizer,
7     HfArgumentParser,
8     PreTrainedTokenizer,
9     TrainingArguments,
10    Trainer,
11 )
12 from transformers.hf_argparser import HfArg
13
14 IGNORE_INDEX = -100
15
16
17 # 用户输入超参数
18 @dataclass
19 class Arguments(TrainingArguments):
20     # 模型结构
21     model_name_or_path: str = HfArg(
22         default=None,
23         help="The model name or path, e.g., `meta-llama/Llama-2-7b-hf`",
24     )
25     # 训练数据集
26     dataset: str = HfArg(
27         default="",
28         help="Setting the names of data file.",
29     )
30     # 上下文窗口大小
31     model_max_length: int = HfArg(
32         default=2048,
33         help="The maximum sequence length",
34     )
35     # 只保存模型参数（不保存优化器状态等中间结果）
36     save_only_model: bool = HfArg(
37         default=True,
38         help="When checkpointing, whether to only save the model, or also
39             ↳ the optimizer, scheduler & rng state.",
40     )
41     # 使用 BF16 混合精度训练
42     bf16: bool = HfArg(
43         default=True,
44         help="Whether to use bf16 (mixed) precision instead of 32-bit.",
45     )
46
47 # 批次化数据，并构建序列到序列损失
48 @dataclass
49 class DataCollatorForSupervisedDataset():
50     tokenizer: PreTrainedTokenizer
51
52     def __call__(self, instances):
53         input_ids, labels = tuple([instance[key] for instance in instances]
54             ↳ for key in ("input_ids", "labels"))
55         input_ids = torch.nn.utils.rnn.pad_sequence(
56             input_ids, batch_first=True,
57             ↳ padding_value=self.tokenizer.pad_token_id
58         )

```



```

57     labels = torch.nn.utils.rnn.pad_sequence(labels, batch_first=True,
58     ↪ padding_value=IGNORE_INDEX)
59     return dict(
60         input_ids=input_ids,
61         labels=labels,
62     )
63
64 def train():
65     # 解析命令行参数
66     parser = HfArgumentParser(Arguments)
67     args = parser.parse_args_into_dataclasses()[0]
68     # 加载分词器
69     tokenizer = AutoTokenizer.from_pretrained(
70         args.model_name_or_path,
71         model_max_length=args.model_max_length,
72         padding_side="right",
73         add_eos_token=False,
74     )
75     # 加载模型，并使用 FlashAttention
76     model = AutoModelForCausalLM.from_pretrained(args.model_name_or_path,
77     ↪ attn_implementation="flash_attention_2")
78     # 初始化训练器、准备训练数据并开始训练
79     kwargs = dict(
80         model=model,
81         args=args,
82         tokenizer=tokenizer,
83         train_dataset=SFTDataset(args, tokenizer),
84         data_collator=DataCollatorForSupervisedDataset(tokenizer),
85     )
86     trainer = Trainer(**kwargs)
87     trainer.train()
88     trainer.save_model(args.output_dir + "/checkpoint-final")
89     trainer.save_state()
90
91
92 if __name__ == "__main__":
93     train()

```

其中，指令微调数据类 `SFTDataset` 的定义如下，`process()` 函数涵盖了指令微调数据的主要处理步骤，包括数据读取、分词、批次化等主要操作，其借鉴了 Alpaca 的构建方法。

```

1 import json
2
3
4 class SFTDataset:
5     IGNORE_INDEX = -100
6     # 定义指令模板格式
7     instruction_template = "\n### Instruction:\n"
8     response_template = "\n### Output:\n"
9     format_template = {
10         "prompt_input": (
11             "Below is an instruction that describes a task, paired with an
12             ↪ input that provides further context. " +

```

```

12         "Write a response that appropriately completes the request." +
13         ↪ instruction_template + "{instruction}" +
14         "{input}" + response_template
15     ),
16     "prompt_no_input": (
17         "Below is an instruction that describes a task. " +
18         "Write a response that appropriately completes the request." +
19         ↪ instruction_template + "{instruction}" +
20         response_template
21     ),
22 }
23
24 def __init__(self, args, tokenizer):
25     self.args = args
26     self.block_size = self.args.model_max_length
27     self.tokenizer = tokenizer
28     self.input_ids, self.labels = self.process(self.tokenizer)
29
30 # 数据集长度
31 def __len__(self):
32     return len(self.input_ids)
33
34 # 获取第 i 条数据
35 def __getitem__(self, i):
36     return dict(input_ids=self.input_ids[i], labels=self.labels[i])
37
38 # 对输入和输出进行分词并标记输出位置
39 def encode_src_tgt(self, s, t, tokenizer):
40     source_id = tokenizer.encode(s,
41     ↪ max_length=tokenizer.model_max_length, truncation=True)
42     tokenizer.add_eos_token = True
43     input_id = tokenizer.encode(s + t,
44     ↪ max_length=tokenizer.model_max_length, truncation=True,
45     return_tensors='pt')[0]
46     tokenizer.add_eos_token = False
47     label = input_id.clone()
48     label[:len(source_id)] = self.IGNORE_INDEX
49     return input_id, label
50
51 # 调用数据集加载、分词、批次化
52 def process(self, tokenizer):
53     input_ids = []
54     labels = []
55     list_data_dict = json.load(open(self.args.dataset))
56
57     for example in list_data_dict:
58         example['response'] = example.pop('output')
59         s = self.format_template["prompt_input"].format_map(example) if
60         ↪ 'input' in example.keys(
61         ) else
62         ↪ self.format_template["prompt_no_input"].format_map(example)
63         t = example['response'].strip()
64         input_id, label = self.encode_src_tgt(s, t, tokenizer)
65         input_ids.append(input_id)
66         labels.append(label)
67     return input_ids, labels

```

为了方便读者了解大模型指令微调的成本，这里使用包含 52K 条指令的 AI-

表 7.1 全量指令微调所需的 A800 GPU 数量、批次大小和微调时间

模型	#GPU	批次大小	时间
LLaMA (7B)	2	8	3.0 小时
LLaMA (13B)	4	8	3.1 小时
LLaMA (30B)	8	4	6.1 小时
LLaMA (65B)	16	2	11.2 小时

paca 数据集，对不同大小的 LLaMA 模型进行了全参数的指令微调实验。表 7.1 中统计了对不同规模的 LLaMA 进行全参数微调所需要的 GPU 数量、批次大小和微调时间。微调实验基于两台 Linux 服务器进行，分别配备了 8 个 A800 (80G) SXM4 GPU (装有 6 个 NVSwitch)。实验中，我们对于不同大小的 LLaMA 模型训练 3 轮，最大序列长度设置为 512，使用数据并行、ZeRO 阶段 3、BF16 和激活重计算技术。注意，这里使用的训练技术与第 6.2 和 6.3 节中的一致，读者也可以使用第 6.4 节中的方法来更细致地估计微调模型需要的时间和显存开销。

7.4.2 指令微调的实验性分析

在微调大模型时，使用不同指令集合微调的大模型在下游任务中往往展现出不同的模型性能。在本节将应用上文的指令微调代码，研究不同类型的指令数据和指令构造策略对于微调大模型的影响。

指令数据集

根据第 7.1 节的讨论，实验中主要考虑以下三种常见类型的指令数据：

- *NLP* 任务数据. 实验采用了目前广泛使用的多任务指令数据集 FLAN v2 [41]。完整的 FLAN v2 集合包含 1,836 个任务和约 20M 个指令实例。
- 日常任务数据. 实验采用了 Vicuna 模型的核心指令数据集 ShareGPT [38]，其中包含了 63K 条真实用户提交的指令和 ChatGPT 对应回复。
- 合成实例数据. 实验采用了合成指令数据集 Alpaca [74]，其中包含了 52K 条指令和相应的输入输出。

考虑到完整的 FLAN v2 数据集非常庞大，实验从中随机抽取了 50K 个实例，以便与其他指令数据集在相似的数量规模上进行公平比较。在实验中会对每种指令集进行性能测试来探究它们各自的效果。

指令改进策略

在第 7.1.3 节中已经介绍了指令数据的合成策略，来作为收集大量用户数据的替代方案。然而，传统的 Alpaca 指令集会存在一些潜在问题，例如指令过于简单或者话题多样性不足等。接下来，我们将基于 Alpaca 指令集，使用现有工作中广泛使用的两种改进策略（详见第 7.1.4 节）对于其进行拓展，并进行对比实验分析。下面介绍这两种实验策略的具体实现。

- 增强指令复杂性. 实验采用了 WizardLM [198] 所提出的 Evol-Instruct 方法逐渐增加指令的复杂性，使用了公开的 WizardLM-70K 指令数据集⁴，这些指令数据是基于 Alpaca 数据集通过上述增强策略进行合成的。

- 增加话题多样性. 实验采用了 YuLan-Chat-3 [73] 所提出的主题多样化方法，利用 ChatGPT 对指令进行重写，并通过特定提示将 Alpaca 数据集的指令适配到 293 个话题。最终，获得了 70K 条指令实例来作为多样化后的数据集。

实验设置

在实验中，我们使用三类指令微调数据集（FLAN v2、ShareGPT 和 Alpaca）和两个拓展的指令集（Alpaca+ 复杂化、Alpaca+ 多样化）来微调 LLaMA-2 模型，通过特定的任务评测来对比不同指令数据集对于模型性能的影响。这里，设置批次大小为 128，学习率恒定为 1×10^{-5} ，总共训练 3 轮，每训练 200 步设置一个存档点，最后选择下游任务表现最佳的存档点来评测该策略的表现。所有指令微调实验均在一台配备 8 个 A800 (80G) 的服务器上完成。

为了更好地评估模型指令微调后的能力，这里主要考虑了两种评测场景，分别是日常对话和 NLP 任务。其中，日常对话评测是基于 AlpacaFarm 的评估数据集开展⁵，其中包含了 800 多个日常生活中的问题和使用 text-davinci-003 模型生成的对应回复。评测使用 ChatGPT 自动对比微调模型和 text-davinci-003 模型的输出质量，计算微调模型的胜率作为评价指标。对于 NLP 任务，我们选择了两个常用评测基准：MMLU [220] 和 BBH [43]（详细介绍见第 12.4 节），均使用准确率来衡量模型表现。

结果与分析

表 7.2 展示了 7B 和 13B 的 LLaMA-2 模型基于不同指令数据的微调结果。下面进行实验结果的分析与讨论。

- 使用与下游任务格式更接近的指令能够带来更大提升. 实验发现 FLAN v2

⁴https://huggingface.co/datasets/victor123/evol_instruct_70k

⁵https://github.com/tatsu-lab/alpaca_farm

表 7.2 基于 LLaMA-2 (7B) 和 (13B) 指令微调的实验结果

模型	指令数据集	指令数量	日常对话	NLP 任务	
			AlpacaFarm	MMLU	BBH
LLaMA-2 (7B)	① FLAN v2	50,000	12.38	50.25	40.63
	② ShareGPT	63,184	55.53	49.66	35.91
	③ Alpaca	52,002	46.58	46.48	36.25
	Alpaca+ 复杂化	70,000	52.92	46.87	35.70
	Alpaca+ 多样化	70,000	52.92	47.52	35.59
LLaMA-2 (13B)	① FLAN v2	50,000	11.58	53.02	45.47
	② ShareGPT	63,184	59.13	56.81	40.80
	③ Alpaca	52,002	48.51	53.89	39.75
	Alpaca+ 复杂化	70,000	55.78	54.85	40.54
	Alpaca+ 多样化	70,000	58.20	55.12	40.26

在 NLP 任务（即 MMLU 和 BBH）上性能优于 ShareGPT 和 Alpaca，然而在日常对话 (AlpacaFarm) 评测中则不如 ShareGPT 和 Alpaca 的效果好。FLAN v2 由传统 NLP 任务（如翻译和阅读理解）的指令数据混合组成，因此能够直接提升模型在 NLP 任务上的效果，但在开放式的日常用户查询则表现较差。相比之下，ShareGPT 包含了真实世界的人类对话，能够更好地引导模型在日常对话任务中去遵循用户指令，但可能不擅长完成传统的 NLP 任务。而 Alpaca 指令集作为日常数据 ShareGPT 的替代方案，在对话任务上相较于 FLAN v2 也有明显的提升，但与 ShareGPT 仍有一定差距。

- 提高指令复杂性和多样性能够促进模型性能的提升。实验发现通过提高 Alpaca 数据集的复杂性和多样性可以有效提升 LLaMA-2 模型在日常对话上的效果，对 NLP 任务也有一定帮助，几乎接近日常对话数据 ShareGPT 的表现。在实验中，使用多样化策略训练的 LLaMA-2 (13B) 在对话任务上的表现提升非常明显，从 48.51 提高到 58.20。此外，提高指令复杂性对模型的表现也有增益，使用第 7.1.3 节中介绍的演化策略后，可以增强模型回复的深度和广度，从而提高对话任务的表现。

- 更大的参数规模有助于提升模型的指令遵循能力。通过对比 LLaMA-2 (7B) 和 LLaMA-2 (13B) 模型在相同指令数据集合上的微调结果，可以明显地看到 LLaMA-2 (13B) 在所有情况下取得了更好的性能表现。这说明扩展模型的参数规模能够加强模型的指令遵循能力。此外，LLaMA-2 (13B) 相比于 LLaMA-2 (7B) 在 NLP 任务上的性能得到了大幅提升，使用 ShareGPT 数据在 MMLU 上的表现从 49.66 提

高到 56.81。这是因为较大的模型通常具有更好的知识利用和推理能力，可以更准确地解决复杂问题 [23, 25]。

7.4.3 LoRA 代码实践与分析

本节将通过示例代码介绍 LoRA 的原理和实现方式，进一步展示如何使用 LoRA 算法端到端地微调一个模型，并且给出相应的资源占用分析。

示例使用代码

下面，首先介绍 LoRA 底层的代码实现，方便读者深入理解其原理；然后介绍 LLMBox 中 LoRA 训练的流程。

```

1 # 继承 PyTorch 的线性变换类
2 class LoRALinear(nn.Linear):
3
4     def __init__(self, in_features, out_features, config, bias=True):
5         super().__init__(in_features, out_features, bias=bias)
6         # 从配置中获取 LoRA 的秩，这决定了低秩矩阵 A 和 B 的大小
7         self.r = config.lora_r
8
9         # 初始化 A，将输入映射到低秩空间 r
10        self.A = nn.Linear(in_features, self.r, bias=False)
11        # 初始化 B，将低秩空间映射回原始输出空间
12        self.B = nn.Linear(self.r, out_features, bias=False)
13
14        # 初始化一个丢弃层，用于在输入传递给 A 之前进行正则化
15        self.dropout = nn.Dropout(p=config.lora_dropout)
16
17        # 使用标准差为 0.02 的正态分布初始化 A 的权重
18        self.A.weight.data.normal_(std=0.02)
19        # B 的权重初始化为零
20        self.B.weight.data.zero_()
21
22    def forward(self, input):
23        # 原始权重对应输出
24        linear_output = F.linear(input, self.weight, self.bias)
25
26        # LoRA 模块对应输出
27        lora_output = self.B(self.A(self.dropout(input)))
28
29        # 将标准线性输出与缩放后的 LoRA 输出相加，得到最终输出
30        return linear_output + lora_output

```

• **LoRA 代码实现** 上述代码定义了一个名为 LoRALinear 的类，通过对 PyTorch 中 nn.Linear 标准线性层的进行扩展，并引入了 LoRA 模块。其中，__init__ 函数中定义了低秩分解矩阵 **A** 和 **B** 和降低的秩数 **R**。此外，LoRALinear 类还包括一个 dropout 层，用于正则化以避免过拟合。forward 函数中计算了原始线性输出

和经缩放的 LoRA 输出之和来作为最终的输出。

- *LoRA* 训练流程. 这里以 LLMBBox 中的参数高效微调代码为例, 其中使用了 Hugging Face 的 PEFT⁶ 参数高效微调代码库, 它支持多种的高效微调方法, 包括 LoRA 和 AdaLoRA、Prefix Tuning、P-Tuning 和 Prompt Tuning 等。下面将介绍如何使用 LLMBBox 对于大语言模型进行 LoRA 微调。

基于第 7.4.1 节中的全参数指令微调代码, LoRA 微调只需添加相应的参数设置即可, 重复的部分在下文中不再展示。具体来说, 首先对 PEFT 库中的 LoRAConfig 类进行实例化, 设置模型架构 task_type、低秩矩阵的维数 r、丢弃率 lora_dropout 等参数, 然后以此参数初始化模型, 即可实现模型的 LoRA 微调。最后将 LoRA 训练的参数与模型参数进行合并, 并保存在本地路径。

```

1  ...
2  # 加载 PEFT 模块相关接口
3  from peft import (
4      LoraConfig,
5      TaskType,
6      AutoPeftModelForCausalLM,
7      get_peft_model,
8  )
9  from transformers.integrations.deepspeed import (
10     is_deepspeed_zero3_enabled,
11     unset_hf_deepspeed_config,
12 )
13 ...
14
15
16 @dataclass
17 class Arguments(TrainingArguments):
18     ...
19     # LoRA 相关超参数
20     lora: Optional[bool] = HfArg(default=False, help="whether to train with
21     ↪ LoRA.")
22
23     lora_r: Optional[int] = HfArg(default=16, help='Lora attention dimension
24     ↪ (the "rank"')
25
26     lora_alpha: Optional[int] = HfArg(default=16, help="The alpha parameter
27     ↪ for Lora scaling.")
28
29     lora_dropout: Optional[float] = HfArg(default=0.05, help="The dropout
30     ↪ probability for Lora layers.")
31
32     ...
33
34 def train():
35     ...

```

⁶<https://github.com/huggingface/peft>

```

34 # 加载 LoRA 配置并初始化 LoRA 模型
35 if args.lora:
36     peft_config = LoraConfig(
37         task_type=TaskType.CAUSAL_LM,
38         r=args.lora_r,
39         lora_alpha=args.lora_alpha,
40         lora_dropout=args.lora_dropout,
41     )
42     model = get_peft_model(model, peft_config)
43     ...
44 # 将 LoRA 参数合并到原始模型中
45 if args.lora:
46     if is_deepspeed_zero3_enabled():
47         unset_hf_deepspeed_config()
48     subdir_list = os.listdir(args.output_dir)
49     for subdir in subdir_list:
50         if subdir.startswith("checkpoint"):
51             print("Merging model in ", args.output_dir + "/" + subdir)
52             peft_model =
53                 ↳ AutoPeftModelForCausalLM.from_pretrained(args.output_dir
54                 ↳ + "/" + subdir)
55             merged_model = peft_model.merge_and_unload()
56             save_path = args.output_dir + "/" + subdir + "-merged"
57             merged_model.save_pretrained(save_path)
58             tokenizer.save_pretrained(save_path)
59 if __name__ == "__main__":
60     train()

```

表 7.3 LoRA 指令微调所需的 A800 (80G) 数量、批次大小和微调时间

模型	#GPU	批次大小	时间
LLaMA (7B)	1	16	2.3 小时
LLaMA (13B)	1	8	3.8 小时
LLaMA (30B)	1	1	10.2 小时
LLaMA (65B)	2	1	26.0 小时

资源占用分析

在计算资源有限的情况下，读者可以选择 LoRA 进行高效的参数微调。为了便于进行比较，我们继续使用 Alpaca 数据集对 LLaMA 进行了 LoRA 微调实验，并统计了微调至少需要的 A800 (80G) 数量、批次大小、微调时间的信息。表 7.3 展示了使用 LoRA 微调 LLaMA 模型所需要消耗的资源。实验中对不同大小的 LLaMA 模型共训练 3 轮，将秩 R 设置为 16，最大序列长度设置为 512。根据第 7.3.1 节中的计算，LoRA 微调至少需要参数量 2 倍大小的显存，因此，对于 7B、13B 和 30B 模型来说，最少只需要一张 A800 (80G) 即可运行，而 65B 模型则至少需要两张。

可以看到，相较于全量微调场景，7B、13B、30B 和 65B 模型需要的显卡数从 2、4、8 和 16（表 7.1）分别降至 1、1、1 和 2，可以大幅减少训练需要的显存量。但是，在微调 30B 和 65B 的模型时，由于模型参数本身占据了绝大多数显存，在表中所给资源情况下批次大小只能设置为 1，导致了训练效率的大幅下降。